

End-Term Report: Functional Planner Kernels for Dagents

Pradyun Devarakonda Dheeraj Rajanala
Course Project for Programming Languages
Roll Number: IMT2022525, IMT2022093

Abstract—This report summarizes the final functional programming contribution in Dagents, a reusable ML automation and data-engineering orchestration framework. The project uses OCaml as a pure functional planner layer inside a larger Python, Java, and containerized service stack. The implemented modules validate source specifications, plan data extraction, infer and validate schemas, evaluate severity-aware quality rules, plan transformations, validate pipeline DAGs, route model families, and render Kubernetes manifests. The result is a polyglot system where functional programming improves the reliability of the decision-making layer without replacing runtime services.

Index Terms—OCaml, functional programming, planning, data engineering, ML orchestration, type systems, Kubernetes

I. Problem

Dagents contains several runtime components: LMA for local source-scoped execution, GMA for aggregate coordination, a pipeline service for JSON workflows, a model service for ML execution, and a core service for service catalog and workload generation. A recurring issue in such systems is that validation and orchestration decisions can become scattered across services.

The functional programming goal was to isolate deterministic decisions into a typed planner layer. In this report, the word “compiler” only means domain-specific translation from one Dagents representation to another. It does not mean a programming-language compiler.

II. Functional Solution

The OCaml workspace is organized around reusable planner modules:

```
bindings/ocaml
|-- lib/common_ir
|-- lib/dataset_compiler
|-- lib/pipeline_compiler
|-- lib/model_router
|-- lib/manifest_compiler
'-- bin/dagentsc.ml
```

A. Typed IR

The `common_ir` module defines algebraic data types for sources, records, schemas, quality operators, pipeline steps, model families, partition strategies, and workload kinds. These types make the valid planning domain explicit.

B. Dataset Planner

The dataset planner validates source specs, creates extraction plans, infers schemas, validates schema contracts, evaluates quality rules, and plans record transforms. Quality evaluation now includes aggregate blocking status based on rule severity.

C. Pipeline Planner

The pipeline planner validates workflow DAGs by rejecting duplicate steps, unknown dependencies, and cycles. It returns a topologically ordered plan and assigns each step to an execution target.

D. Model Router and Manifest Renderer

The model router maps dataset profiles and task types to candidate model families and packaging modes. The manifest renderer turns typed workload specs into Kubernetes Deployment, Job, CronJob, Service, and ConfigMap YAML.

III. Application Integration

The larger Dagents app stays polyglot. Python services own FastAPI endpoints, data access, pipeline execution, and model training. OCaml owns pure planning. The boundary is the `dagentsc` CLI, which exchanges JSON payloads with services.

```
dagentsc dataset quality evaluate \
--records docs/demo/inputs/records-orders.json \
--rules docs/demo/inputs/quality-rules-orders.json
```

The demo package under `docs/demo` shows both the full service stack and the functional layer. The full-stack script exercises health, topology, pipeline, model, LMA/GMA, and workload flows. The functional-layer script shows the OCaml planner decisions directly.

IV. Testing Evidence

The OCaml tests cover positive and negative planner behavior:

- source validation and invalid source rejection;
- time-window and hash partition planning;
- schema contract success and failure;
- severity-aware quality reporting;
- transform planning and application;
- pipeline ordering, duplicate rejection, unknown dependency rejection, and cycle rejection;

- model routing and manifest rendering.

The main verification commands are:

```
opam exec -- dune test
opam exec -- dune build ./bin/dagentsc.exe
bash docs/demo/run_functional_layer_demo.sh
bash docs/demo/run_demo_quick.sh
```

Python wrapper tests mock subprocess calls to verify the service boundary without requiring a live `dagentsc` executable.

V. Programming Languages Takeaways

The project demonstrates practical use of functional programming in a real application:

- algebraic data types model allowed domain states;
- pattern matching makes planning cases explicit;
- pure functions make outputs deterministic;
- typed JSON boundaries allow safe polyglot integration;
- OCaml improves the reliability of planning without replacing Python ML execution.

VI. Limitations

The OCaml layer deliberately avoids runtime I/O. It does not connect to Postgres, MongoDB, object storage, Kubernetes, or PyTorch. This is a design tradeoff: the planner remains pure and testable, but runtime services still need robust adapters and operational handling.

VII. Conclusion

Dagents uses functional programming as a targeted engineering tool. OCaml provides a typed planner layer for validation, routing, transformation planning, and manifest rendering. The full app remains practical and polyglot, while the most deterministic orchestration decisions become easier to test, explain, and extend.

References

- [1] R. Milner, “A theory of type polymorphism in programming,” *Journal of Computer and System Sciences*, vol. 17, no. 3, pp. 348–375, 1978.
- [2] X. Leroy, D. Doligez, A. Frisch, J. Garrigue, D. Remy, and J. Vouillon, *The OCaml System: Documentation and User’s Manual*. OCaml.org. [Online]. Available: <https://ocaml.org/manual/>
- [3] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” in *Proc. 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2004, pp. 137–150.
- [4] The Kubernetes Authors, “Workloads,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/>